

## Lab 8

1. Write an OpenGL program to perform 3D Translation of a Cube and display both the original and translated cube.

```
#include <GL/glut.h>

float tx = 1.0f, ty = 0.5f, tz = -1.0f;

void drawCube() {
    glutSolidCube(1.0);
}

void drawWireCube() {
    glutWireCube(1.0);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    gluLookAt(3.0, 3.0, 7.0,
              0.0, 0.0, 0.0,
              0.0, 1.0, 0.0);

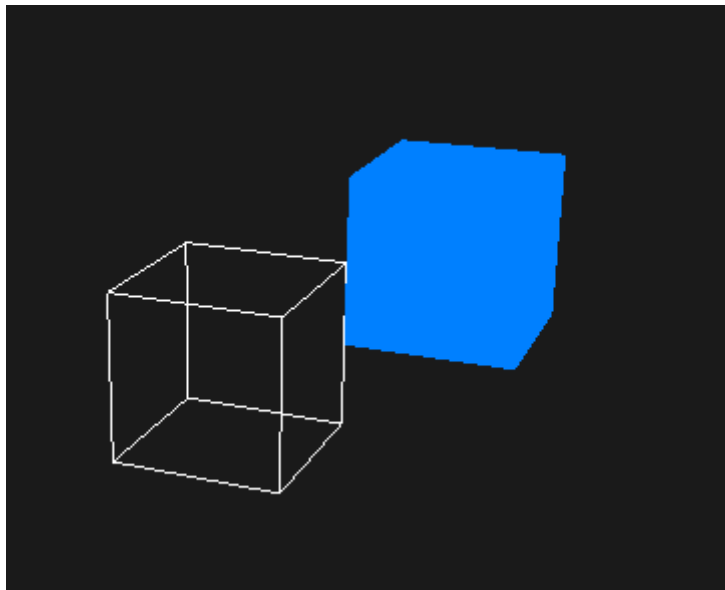
    glColor3f(1.0, 1.0, 1.0);
    drawWireCube();
    glPushMatrix();
    glTranslatef(tx, ty, tz);
    glColor3f(0.0, 0.5, 1.0);
    drawCube();
    glPopMatrix();
    glutSwapBuffers();
}

void init() {
    glEnable(GL_DEPTH_TEST);
    glClearColor(0.1, 0.1, 0.1, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
```

```

gluPerspective(45.0, 1.33, 1.0, 20.0);
glMatrixMode(GL_MODELVIEW);
}
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("OpenGL 3D Translation - Original vs Translated");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```



2. Program to perform 3D Scaling of a Pyramid and display both the original and scaled object.

```
#include <GL/glut.h>
```

```
float sx = 1.5f, sy = 1.5f, sz = 1.5f;
```

```
void drawPyramidBoundary() {
```

```
glBegin(GL_LINE_LOOP);  
glVertex3f(0.0f, 1.0f, 0.0f);  
glVertex3f(-1.0f, -1.0f, 1.0f);  
glVertex3f(1.0f, -1.0f, 1.0f);  
glEnd();
```

```
glBegin(GL_LINE_LOOP);  
glVertex3f(0.0f, 1.0f, 0.0f);  
glVertex3f(1.0f, -1.0f, 1.0f);  
glVertex3f(1.0f, -1.0f, -1.0f);  
glEnd();
```

```
glBegin(GL_LINE_LOOP);  
glVertex3f(0.0f, 1.0f, 0.0f);  
glVertex3f(1.0f, -1.0f, -1.0f);  
glVertex3f(-1.0f, -1.0f, -1.0f);  
glEnd();
```

```
glBegin(GL_LINE_LOOP);  
glVertex3f(0.0f, 1.0f, 0.0f);  
glVertex3f(-1.0f, -1.0f, -1.0f);  
glVertex3f(-1.0f, -1.0f, 1.0f);  
glEnd();
```

```
glBegin(GL_LINE_LOOP);  
glVertex3f(-1.0f, -1.0f, 1.0f);  
glVertex3f(1.0f, -1.0f, 1.0f);  
glVertex3f(1.0f, -1.0f, -1.0f);  
glVertex3f(-1.0f, -1.0f, -1.0f);  
glEnd();
```

```
}
```

```

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    gluLookAt(4.0, 4.0, 8.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);

    glColor3f(1.0, 1.0, 1.0);
    drawPyramidBoundary();

    glPushMatrix();
        glScalef(sx, sy, sz);
        glColor3f(0.0, 1.0, 1.0);
        drawPyramidBoundary();
    glPopMatrix();

    glutSwapBuffers();
}

void init() {
    glEnable(GL_DEPTH_TEST);
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, 1.33, 1.0, 20.0);
    glMatrixMode(GL_MODELVIEW);
}

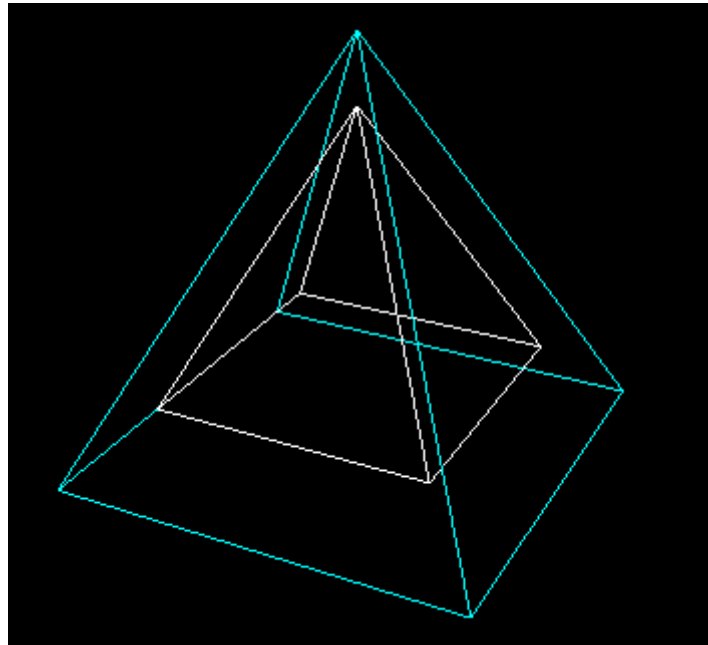
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("3D Scaling - Boundary Only");
}

```

```

init();
glutDisplayFunc(display);
glutMainLoop();
return 0;
}

```



3. Program to perform 3D Rotation about the X-axis and display the rotated object along with the original.

```
#include <GL/glut.h>
```

```
float angle = 45.0f;
```

```

void drawWireCube() {
    glutWireCube(1.0);
}

```

```

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    gluLookAt(3.0, 3.0, 7.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);

```

```
glColor3f(1.0, 1.0, 1.0);
```

```
drawWireCube();
```

```
glPushMatrix();
```

```
    glRotatef(angle, 1.0, 0.0, 0.0);
```

```
    glColor3f(1.0, 0.0, 0.0);
```

```
    drawWireCube();
```

```
glPopMatrix();
```

```
glutSwapBuffers();
```

```
}
```

```
void init() {
```

```
    glEnable(GL_DEPTH_TEST);
```

```
    glClearColor(0.0, 0.0, 0.0, 1.0);
```

```
    glMatrixMode(GL_PROJECTION);
```

```
    glLoadIdentity();
```

```
    gluPerspective(45.0, 1.33, 1.0, 20.0);
```

```
    glMatrixMode(GL_MODELVIEW);
```

```
}
```

```
int main(int argc, char** argv) {
```

```
    glutInit(&argc, argv);
```

```
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
```

```
    glutInitWindowSize(800, 600);
```

```
    glutCreateWindow("3D Rotation X-Axis - Boundary Only");
```

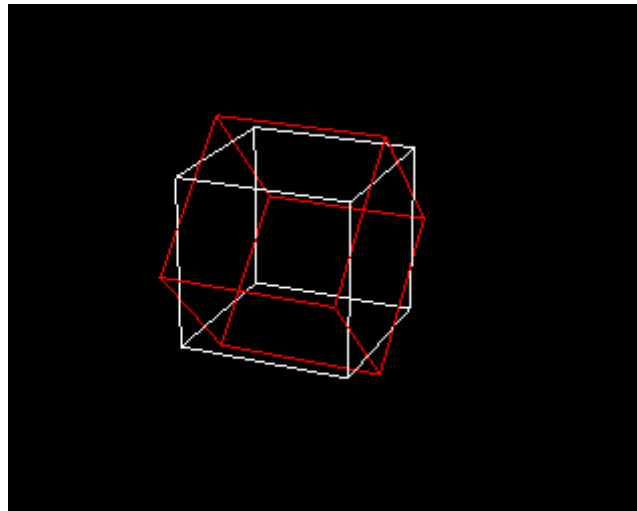
```
    init();
```

```
    glutDisplayFunc(display);
```

```
    glutMainLoop();
```

```
    return 0;
```

```
}
```



4. Program to implement the Cohen–Sutherland Line Clipping Algorithm to clip a line against a rectangular clipping window.

```
#include <GL/glut.h>
#include <iostream>
using namespace std;

float xmin, ymin, xmax, ymax;

const int LEFT = 1, RIGHT = 2, BOTTOM = 4, TOP = 8;

int getCode(float x, float y) {
    int code = 0;

    if (x < xmin) code |= LEFT;
    if (x > xmax) code |= RIGHT;
    if (y < ymin) code |= BOTTOM;
    if (y > ymax) code |= TOP;

    return code;
}

void cohenClip(float x1, float y1, float x2, float y2) {
    int code1 = getCode(x1, y1);
    int code2 = getCode(x2, y2);

    while (true) {
        if (!(code1 | code2)) {
            glBegin(GL_LINES);
            glVertex2f(x1, y1);
            glVertex2f(x2, y2);
            glEnd();
            break;
        }
        else if (code1 & code2) {
            break;
        }
    }
}
```

```

    }
    else {
        float x, y;
        int codeOut = code1 ? code1 : code2;

        if (codeOut & TOP) {
            x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1);
            y = ymax;
        }
        else if (codeOut & BOTTOM) {
            x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1);
            y = ymin;
        }
        else if (codeOut & RIGHT) {
            y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1);
            x = xmax;
        }
        else if (codeOut & LEFT) {
            y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1);
            x = xmin;
        }

        if (codeOut == code1) {
            x1 = x;
            y1 = y;
            code1 = getCode(x1, y1);
        } else {
            x2 = x;
            y2 = y;
            code2 = getCode(x2, y2);
        }
    }
}

```

```

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1, 1, 1);
    glBegin(GL_LINE_LOOP);
        glVertex2f(xmin, ymin);
        glVertex2f(xmax, ymin);
        glVertex2f(xmax, ymax);
        glVertex2f(xmin, ymax);
    glEnd();

    glColor3f(0, 1, 0);
    cohenClip(50, 50, 450, 450);
    cohenClip(50, 400, 450, 100);

    glFlush();
}

```

```
void init() {
    glClearColor(0, 0, 0, 1);
    gluOrtho2D(0, 500, 0, 500);
}

int main(int argc, char** argv) {
    cout << "Enter xmin ymin: ";
    cin >> xmin >> ymin;
    cout << "Enter xmax ymax: ";
    cin >> xmax >> ymax;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Cohen-Sutherland Line Clipping");

    init();
    glutDisplayFunc(display);

    glutMainLoop();
    return 0;
}
```

